

Ketentuan Tugas Pendahuluan

- Untuk soal teori **JAWABAN DIKETIK DENGAN RAPI** dan untuk soal algoritma **SERTAKAN SCREENSHOOT CODINGAN DAN HASIL OUTPUT**.
- Deadline pengumpulan TP Modul 5 adalah **Senin, 4 November 2024 pukul 06.00 WIB**.
- **TIDAK ADA TOLERANSI KETERLAMBATAN, TERLAMBAT ATAU TIDAK MENGUMPULKAN TP ONLINE MAKA DIANGGAP TIDAK MENGERJAKAN**.
- **DILARANG PLAGIAT (PLAGIAT = E)**.
- Kerjakan TP dengan jelas agar dapat dimengerti.
- Untuk setiap soal nama fungsi atau prosedur **WAJIB** menyertakan **NIM**, contoh: **insertFirst_130122xxxx**.
- File diupload di LMS menggunakan format PDF dengan ketentuan : **TP_MODX_NIM_KELAS.pdf**

Contoh:

```
int searchNode_130122xxxx (List L, int X);
```

CP:

- Raihan (089638482851)

- Kayyisa (085105303555)
- Abiya (082127180662)
- Rio (081210978384)

SELAMAT MENGERJAKAN^^

LAPORAN PRAKTIKUM

MODUL 6

“ Double Linked List”



Disusun Oleh:

Shilfi Habibah - 2311104002

S1SE07-01

Dosen :

Yudha Islami Sulistya

PROGRAM STUDI S1 SOFTWARE ENGINEERING
FAKULTAS INFORMATIKA
INSTITUT TEKNOLOGI TELKOM PURWOKERTO
2024

SOAL TP

Soal 1: Menambahkan Elemen di Awal dan Akhir DLL

Deskripsi Soal:

Buatlah program yang mengizinkan pengguna menambahkan elemen ke dalam Doubly Linked List di awal dan di akhir list.

Instruksi

1. Implementasikan fungsi `insertFirst` untuk menambahkan elemen di awal list.
2. Implementasikan fungsi `insertLast` untuk menambahkan elemen di akhir list.
3. Tampilkan seluruh elemen dalam list dari depan ke belakang setelah penambahan dilakukan.

Contoh Output:

- Input: Masukkan elemen pertama = 10
- Input: Masukkan elemen kedua di awal = 5
- Input: Masukkan elemen ketiga di akhir = 20

Output:

- DAFTAR ANGGOTA LIST: 5 <-> 10 <-> 20

Code:

```

#include <iostream>
using namespace std;

// Struktur node pada Doubly Linked List
struct Node {
    int data;
    Node* next;
    Node* prev;
};

// Fungsi untuk membuat node baru
Node* createNode(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = nullptr;
    newNode->prev = nullptr;
    return newNode;
}

// Fungsi untuk menambahkan elemen di awal list
void insertFirst_2311104002(Node*& head, int value) {
    Node* newNode = createNode(value);
    if (head == nullptr) {
        head = newNode;
    } else {
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }
}

// Fungsi untuk menambahkan elemen di akhir list
void insertLast_2311104002(Node*& head, int value) {
    Node* newNode = createNode(value);
    if (head == nullptr) {
        head = newNode;
    } else {
        Node* temp = head;
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }
}

// Fungsi untuk mencetak list dari depan ke belakang
void printList(Node* head) {
    if (head == nullptr) {
        cout << "List kosong." << endl;
        return;
    }

    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data;
        if (temp->next != nullptr) {
            cout << " <-> ";
        }
        temp = temp->next;
    }
    cout << endl;
}

```

```

int main() {
    Node* head = nullptr; // List kosong pada awalnya
    int firstElement, secondElement, thirdElement;

    // Memasukkan elemen pertama di awal list
    cout << "Masukkan elemen pertama: ";
    cin >> firstElement;
    insertFirst_2311104002(head, firstElement);

    // Memasukkan elemen kedua di awal list
    cout << "Masukkan elemen kedua di awal: ";
    cin >> secondElement;
    insertFirst_2311104002(head, secondElement);

    // Memasukkan elemen ketiga di akhir list
    cout << "Masukkan elemen ketiga di akhir: ";
    cin >> thirdElement;
    insertLast_2311104002(head, thirdElement);

    // Menampilkan seluruh elemen dalam list
    cout << "DAFTAR ANGGOTA LIST: ";
    printList(head);

    return 0;
}

```

- Fungsi **Node** : data (nilai integer dari node) , next (pointer ke ode berikutnya) , prev (pointer ke node sebelumnya)
- Fungsi createNode digunakan untuk membuat dan menginisialisasi node baru menggunakan data yang diberikan serta mengambil pointer pada node tersebut.
- Terdapat fungsi insertFirst untuk menambahkan elemen baru pada awal list, jika list kosong maka elemen menjadi head, Jika tidak kosong menjadi node berikutnya
- Ada juga insertLast yaitu menambahkan elemen baru pada akhir list, Jika list kosong elemen menjadi head , Jika tidak maka akan menelusuri list hingga node terakhir setelah itu tambah node baru.
- Fungsi printList yaitu untuk mencetak elemen list dari depan ke belakang dengan format "data1 <-> data2 <-> data3...".

Output:

```

Masukkan elemen pertama: 10
Masukkan elemen kedua di awal: 5
Masukkan elemen ketiga di akhir: 20
DAFTAR ANGGOTA LIST: 5 <-> 10 <-> 20

Process returned 0 (0x0)   execution time : 4.392 s
Press any key to continue.

```

- Program meminta inputan dari pengguna agar memasukkan elemen pertama, kedua, ketiga , kemudian menampilkan seluruh elemen dalam list dalam urutan yang sesuai.

Soal 2: Menghapus Elemen di Awal dan Akhir DLL

Deskripsi Soal:

Buatlah program yang memungkinkan pengguna untuk menghapus elemen pertama dan elemen terakhir dalam Doubly Linked List.

Instruksi

1. Implementasikan fungsi `deleteFirst` untuk menghapus elemen pertama.
2. Implementasikan fungsi `deleteLast` untuk menghapus elemen terakhir.
3. Tampilkan seluruh elemen dalam list setelah penghapusan dilakukan.

Contoh Output

- Input: Masukkan elemen pertama = 10
- Input: Masukkan elemen kedua di akhir = 15
- Input: Masukkan elemen ketiga di akhir = 20
- Hapus elemen pertama dan terakhir.

Output

- DAFTAR ANGGOTA LIST SETELAH PENGHAPUSAN: 15

Code:

```

#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* prev;
    Node* next;
};

class DoublyLinkedList {
private:
    Node* head;

public:
    DoublyLinkedList() {
        head = nullptr;
    }

    // Menambahkan elemen di akhir list
    void append(int data) {
        Node* newNode = new Node();
        newNode->data = data;
        newNode->next = nullptr;

        if (head == nullptr) { // Jika list kosong
            newNode->prev = nullptr;
            head = newNode;
        } else {
            Node* temp = head;
            while (temp->next != nullptr) {
                temp = temp->next;
            }
            temp->next = newNode;
            newNode->prev = temp;
        }
    }

    // Menghapus elemen pertama
    void deleteFirst_2311104002() {
        if (head == nullptr) {
            cout << "List kosong, tidak ada elemen yang bisa dihapus.\n";
            return;
        }
        Node* temp = head;
        head = head->next;
        if (head != nullptr) {
            head->prev = nullptr;
        }
        delete temp;
    }

    // Menghapus elemen terakhir
    void deleteLast_2311104002() {
        if (head == nullptr) {
            cout << "List kosong, tidak ada elemen yang bisa dihapus.\n";
            return;
        }
        Node* temp = head;
        if (temp->next == nullptr) { // Hanya satu elemen dalam list
            head = nullptr;
            delete temp;
            return;
        }
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->prev->next = nullptr;
        delete temp;
    }
}

```



```

// Menampilkan elemen-elemen dalam list
void display() {
    if (head == nullptr) {
        cout << "List kosong.\n";
        return;
    }
    Node* temp = head;
    cout << "DAFTAR ANGGOTA LIST SETELAH PENGHAPUSAN: ";
    while (temp != nullptr) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

};

int main() {
    DoublyLinkedList dll;
    dll.append(10);
    dll.append(15);
    dll.append(20);

    // Hapus elemen pertama dan terakhir
    dll.deleteFirst_2311104002();
    dll.deleteLast_2311104002();

    // Tampilkan elemen setelah penghapusan
    dll.display();

    return 0;
}

```

- Terdapat beberapa program di mulai dari append yaitu menambahkan elemen baru pada akhir list , deleteFirst untuk menghapus elemen pertama dari list, deleteLast untuk menghapus elemen terakhir dari list dan display untuk menampilkan seluruh elemen dalam list.

Output:

```

DAFTAR ANGGOTA LIST SETELAH PENGHAPUSAN: 15
Process returned 0 (0x0)   execution time : 0.295 s
Press any key to continue.

```

- dari output kita bisa menambahkan elemen lain atau megubah elemen yang dimasukkan untuk menguji fungsionalitas penghapusan pertama dan terakhir.

Soal 3: Menampilkan Elemen dari Depan ke Belakang dan Sebaliknya

Deskripsi Soal:

Buatlah program yang memungkinkan pengguna memasukkan beberapa elemen ke dalam Doubly Linked List. Setelah elemen dimasukkan, tampilkan seluruh elemen dalam list dari depan ke belakang, kemudian dari belakang ke depan.

Instruksi

1. Implementasikan fungsi untuk menampilkan elemen dari depan ke belakang.
2. Implementasikan fungsi untuk menampilkan elemen dari belakang ke depan.
3. Tambahkan 4 elemen ke dalam list dan tampilkan elemen tersebut dalam dua arah.

Contoh Input:

- Input: Masukkan 4 elemen secara berurutan: 1, 2, 3, 4

Output:

- Daftar elemen dari depan ke belakang: 1 <-> 2 <-> 3 <-> 4
- Daftar elemen dari belakang ke depan: 4 <-> 3 <-> 2 <-> 1

Code:

```

#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* prev;
    Node* next;
};

class DoublyLinkedList {
private:
    Node* head;

public:
    DoublyLinkedList() {
        head = nullptr;
    }

    // Menambahkan elemen di akhir list
    void append(int data) {
        Node* newNode = new Node();
        newNode->data = data;
        newNode->next = nullptr;

        if (head == nullptr) { // Jika list kosong
            newNode->prev = nullptr;
            head = newNode;
        } else {
            Node* temp = head;
            while (temp->next != nullptr) {
                temp = temp->next;
            }
            temp->next = newNode;
            newNode->prev = temp;
        }
    }
}

```

```

// Menampilkan elemen dari depan ke belakang
void displayForward() {
    if (head == nullptr) {
        cout << "List kosong.\n";
        return;
    }
    Node* temp = head;
    cout << "Daftar elemen dari depan ke belakang: ";
    while (temp != nullptr) {
        cout << temp->data;
        if (temp->next != nullptr) {
            cout << " <-> ";
        }
        temp = temp->next;
    }
    cout << endl;
}

// Menampilkan elemen dari belakang ke depan
void displayBackward() {
    if (head == nullptr) {
        cout << "List kosong.\n";
        return;
    }
    Node* temp = head;
    // Mencari node terakhir
    while (temp->next != nullptr) {
        temp = temp->next;
    }

    cout << "Daftar elemen dari belakang ke depan: ";
    while (temp != nullptr) {
        cout << temp->data;
        if (temp->prev != nullptr) {
            cout << " <-> ";
        }
        temp = temp->prev;
    }
    cout << endl;
}

};

int main() {
    DoublyLinkedList dll;

    // Menambahkan 4 elemen ke dalam list
    dll.append(1);
    dll.append(2);
    dll.append(3);
    dll.append(4);

    // Tampilkan elemen dari depan ke belakang
    dll.displayForward();

    // Tampilkan elemen dari belakang ke depan
    dll.displayBackward();

    return 0;
}

```

- Terdapat beberapa program mulai dari append yaitu berfungsi menambahkan elemen baru di akhir Doubly Linked List, displayForward berfungsi menampilkan elemen-elemen dari depan ke belakang dengan simbol "<->" diantara elemen dan displayBackward berfungsi menampilkan elemen-elemen dari belakang ke depan dengan simbol "<->" diantara elemen.

Output:

```
Daftar elemen dari depan ke belakang: 1 <-> 2 <-> 3 <-> 4
Daftar elemen dari belakang ke depan: 4 <-> 3 <-> 2 <-> 1

Process returned 0 (0x0)   execution time : 0.106 s
Press any key to continue.
```

- Dari output akan menampilkan elemen dalam urutan yang benar sesuai dengan permintaan, bisa juga menambahkan elemen lain dari untuk menguji program lebih lanjut

Latihan (Modul)

1. Buatlah ADT Double Linked list sebagai berikut di dalam file “doublelist.h”:

```
Type infotype : kendaraan <
  nopol : string
  warna : string
  thnBuat : integer
>
Type address : pointer to ElmList
Type ElmList <
  info : infotype
  next : address
  prev : address
>
Type List <
  First : address
  Last : address
>
prosedur CreateList( in/out L : List )
fungsi alokasi( x : infotype ) : address
prosedur dealokasi( in/out P : address )
prosedur printInfo( in L : List )
prosedur insertLast( in/out L : List, in P : address )
```

Buatlah implementasi ADT Double Linked list pada file “doublelist.cpp” dan coba hasil implementasi ADT pada file “main.cpp”.

Code :

```

#ifndef DOUBLELIST_H
#define DOUBLELIST_H

#include <iostream>
#include <string>
using namespace std;

struct kendaraan {
    string nopol;
    string warna;
    int thnBuat;
};

typedef kendaraan infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
    address prev;
};

struct List {
    address First;
    address Last;
};

void CreateList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void printInfo(List L);
void insertLast(List &L, address P);
address findElm(List L, string nopol);
void deleteFirst(List &L, address &P);
void deleteLast(List &L, address &P);
void deleteAfter(address Prec, address &P);

#endif

```

singlelist.h

```

#include "doublelist.h"

// Membuat list kosong
void CreateList(List &L) {
    L.First = nullptr;
    L.Last = nullptr;
}

// Alokasi elemen baru
address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = nullptr;
    P->prev = nullptr;
    return P;
}

// Dealokasi elemen
void dealokasi(address &P) {
    delete P;
}

// Menampilkan seluruh elemen list
void printInfo(List L) {
    address P = L.First;
    int i = 1;
    while (P != nullptr) {
        cout << "Nomor Polisi : " << P->info.nopol << endl;
        cout << "Warna : " << P->info.warna << endl;
        cout << "Tahun : " << P->info.thnBuat << endl;
        P = P->next;
    }
}

// Menambahkan elemen di akhir list
void insertLast(List &L, address P) {
    if (L.First == nullptr) {
        L.First = P;
        L.Last = P;
    }
}

```

```

    } else {
        P->prev = L.Last;
        L.Last->next = P;
        L.Last = P;
    }
}

// Mencari elemen berdasarkan nomor polisi
address findElm(List L, string nopol) {
    address P = L.First;
    while (P != nullptr) {
        if (P->info.nopol == nopol) {
            return P;
        }
        P = P->next;
    }
    return nullptr;
}

// Menghapus elemen pertama
void deleteFirst(List &L, address &P) {
    if (L.First != nullptr) {
        P = L.First;
        L.First = L.First->next;
        if (L.First != nullptr) {
            L.First->prev = nullptr;
        } else {
            L.Last = nullptr;
        }
        P->next = nullptr;
    }
}

// Menghapus elemen terakhir
void deleteLast(List &L, address &P) {
    if (L.Last != nullptr) {
        P = L.Last;
        L.Last = L.Last->prev;
        if (L.Last != nullptr) {
            L.Last->next = nullptr;
        } else {
            L.First = nullptr;
        }
        P->prev = nullptr;
    }
}

// Menghapus elemen setelah elemen tertentu
void deleteAfter(address Prec, address &P) {
    if (Prec != nullptr && Prec->next != nullptr) {
        P = Prec->next;
        Prec->next = P->next;
        if (P->next != nullptr) {
            P->next->prev = Prec;
        }
        P->next = nullptr;
        P->prev = nullptr;
    }
}

```



```

#include "singlelist.h"

int main() {
    List L;
    address P1, P2, P3, P4, P5 = nullptr;

    // Membuat list kosong
    createList(L);

    // Menambahkan elemen-elemen ke dalam list
    P1 = alokasi(2);
    insertFirst(L, P1);

    P2 = alokasi(0);
    insertFirst(L, P2);

    P3 = alokasi(8);
    insertFirst(L, P3);

    P4 = alokasi(12);
    insertFirst(L, P4);

    P5 = alokasi(9);
    insertFirst(L, P5);

    // Menampilkan list
    cout << "Elemen dalam list: ";
    printInfo(L);

    // Mencari elemen dengan info 8
    address found = findElm(L, 8);
    if (found != nullptr) {
        cout << "Elemen dengan info 8 ditemukan." << endl;
    } else {
        cout << "Elemen dengan info 8 tidak ditemukan." << endl;
    }

    // Menghitung total info dari semua elemen
    int total = totalInfo(L);
    cout << "Total nilai elemen dalam list: " << total << endl;

    return 0;
}

```

main.cpp

- ADT list berfungsi menyimpan elemen-elemen kendaraan dengan atribut nomor polisi, warna, dan tahun pembuatan
- Beberapa fungsi utama mulai dari CreateList berfungsi list kosong, insertLast berfungsi menambahkan elemen ke akhir list, printInfo berfungsi menampilkan seluruh elemen dalam list, findElm berfungsi mencari elemen berdasarkan nomor polisi, deleteFirst; deleteLast; deleteAfter berfungsi menghapus elemen dari list

Output:

```
DATA LIST 1:
Nomor Polisi : D001
Warna       : hitam
Tahun       : 90
Nomor Polisi : D003
Warna       : putih
Tahun       : 70
Nomor Polisi : D004
Warna       : kuning
Tahun       : 90

Kendaraan ditemukan:
Nomor Polisi : D001
Warna       : hitam
Tahun       : 90

Data dengan nomor polisi D003 berhasil dihapus.

DATA LIST 1 SETELAH PENGHAPUSAN:
Nomor Polisi : D001
Warna       : hitam
Tahun       : 90
Nomor Polisi : D004
Warna       : kuning
Tahun       : 90
```

- Program ini implementasi dari Double Linked List menggunakan prosedur dasar untuk menambahkan beberapa kendaraan dalam list, mencari kendaraan berdasarkan nomor polisi dan menghapus kendaraan dengan nomor polisi tertentu.